



BOOTCAMP FRONTEND & REACT · SEMANA 3 DE 8

ES6+

Clase 8 — JavaScript moderno: la misma tienda, escrita como se escribe hoy · template literals, desestructuración, spread, rest y módulos

L · M · V

Proyecto TechCart

STEER
RC
LA

HOY NO HAY IDEAS NUEVAS: HAY UNA FORMA NUEVA DE ESCRIBIRLAS

Agenda

1. Módulos: un archivo, un trabajo

export · import · type="module" · el reparto en cuatro archivos

2. Template literals

comillas invertidas · \${ } · texto de varias líneas

3. Desestructuración

objetos, arrays, anidada y en el parámetro

4. Spread: copiar sin mutar

arrays, objetos, al llamar · y la copia superficial

5. Rest

los mismos tres puntos, haciendo lo contrario

6. Los atajos que acompañan

abreviadas · calculadas · defaults · ?. y ??

7. Todo junto

el resumen de la tienda, con los números de ayer

8. Cierre

errores frecuentes, resumen, 5 tareas y anticipo

 **Receso de 15 min** después de la desestructuración · hoy la página se abre con **servidor**, no con doble clic

Dos de memoria

¿Qué devuelve `filter` si ningún elemento cumple?

¿Por qué a `sort` le poníamos `slice()` adelante?

Cuatro símbolos que están en **todo** el código moderno

`` ``

Template literals

armar textos

`{ }`

Desestructuración

abrir un objeto

`...`

Spread y rest

copiar y juntar

`import`

Módulos

partir el código

Todo esto es **ES6 en adelante**, del 2015 en adelante. ES6 y ES2015 son exactamente lo mismo.

JS/MAIN.JS DE LA CLASE 7 · 347 LÍNEAS · HACE DE TODO

Un archivo que crece, **tres problemas**

- 1 Encontrar algo cuesta.** El catálogo, las cuentas, el formateo y las pruebas, todo mezclado en el mismo archivo.
- 2 Con varios `<script>`, el orden importa.** Si el segundo usa algo del primero, el primero tiene que estar arriba. Cambiar dos líneas de lugar rompe la página.
- 3 Todo vive en el ámbito global.** Dos archivos que declaren `total` se pisan el valor, y nadie avisa.

Los módulos arreglan los tres: **cada archivo tiene su propio ámbito** y declara de qué otros depende, así que el orden lo resuelve el navegador.

LA REGLA QUE YA CONOCEN DE JAVA: UN ARCHIVO, UN TRABAJO

TechCart, repartida en **cuatro**

datos.js

El catálogo. Qué vende la tienda, y nada más. No calcula ni arma textos.

formato.js

Cómo se muestra un precio. Un solo trabajo, y por eso estrena `export default`.

tienda.js

Las cuentas: IGV, envío, descuentos, búsqueda y resumen. Recibe los productos por parámetro.

main.js

El archivo de **entrada**. El único que **importa**. Junta las piezas y muestra el resultado.

Los tres primeros no importan nada: **se dejan usar**. Las dependencias apuntan para un solo lado.

EXPORT = PUBLIC · LO QUE NO SE EXPORTA SE QUEDA ADENTRO DEL ARCHIVO

export e import

TIENDA.JS – LO DEJA SALIR

```
export const IGV = 0.18;  
  
// sin export: privado del archivo  
const REDONDEO = 2;
```



MAIN.JS – LO PIDE

```
import { IGV }  
from "./tienda.js";
```

export va delante de un `const`, un `let` o una `function`. Lo que no lo lleva existe y funciona, pero desde afuera nadie lo ve.

LA DIFERENCIA ES DE INTENCIÓN, NO DE POTENCIA

Con nombre, o por defecto

export const

```
export const MONEDA = "$";  
import { MONEDA } from "./formato.js";
```

Varios por archivo. Se importa **entre llaves** y con el **nombre exacto**. Para archivos que ofrecen varias piezas.

export default

```
export default formatearPrecio;  
import formatearPrecio from  
"./formato.js";
```

Uno solo por archivo. Se importa **sin llaves** y el nombre **lo elige quien importa**. Para cuando el archivo **es** una cosa.

Un archivo puede tener **las dos a la vez**, y es lo normal: el protagonista por defecto y las piezas que lo acompañan con nombre.

LOS IMPORT VAN SIEMPRE ARRIBA DEL TODO DEL ARCHIVO

Las tres formas de **import**

```
// con llaves: los exports CON NOMBRE, con el nombre exacto
import { IGV, ENVIO_GRATIS_DESDE } from "./tienda.js";

// sin llaves: el export POR DEFECTO, con el nombre que yo elija
import formatearPrecio from "./formato.js";

// las dos juntas: el por defecto primero, las llaves después
import formatearPrecio, { MONEDA } from "./formato.js";
```

La ruta lleva . / adelante y . js al final, siempre. El navegador no adivina archivos: pide exactamente lo que se le dice. Si escriben ". /datos", busca un archivo llamado datos sin extensión y no lo encuentra.

LA PIEZA QUE FALTA ESTÁ EN EL HTML

type="module"

```
<script type="module" src="./js/main.js"></script>
```

1 · habilita

Sin esto, `import` y `export` son un error de sintaxis.

2 · modo estricto

Siempre, sin escribir `"use strict"`. Reglas más severas.

3 · diferido

Un módulo ya viene diferido de fábrica: la palabra `defer` sobra.

Y exige servidor. Con doble clic la página se abre con `file://` y el navegador bloquea los módulos: `blocked by CORS policy`. Desde hoy, **Live Server**.

COMILLAS INVERTIDAS ` ` · A LA IZQUIERDA DEL 1 O AL LADO DE LA P

Template literals

ANTES · PEGAR PEDAZOS CON +

```
"Hola " + nombre + ", tu total es " + total
```

HOY · UN TEXTO CON HUECOS

```
`Hola ${nombre}, tu total es ${total}`
```

Adentro de `${ }` cabe **cualquier expresión**: una cuenta, una llamada a función, un ternario. Lo que no cabe es un `if` ni un `for`.

Con comillas **normales**, ``${ }` se imprime tal cual y **no hay ningún error**. Mirar siempre qué comillas son.

CON " " EL SALTO DE LÍNEA SE ESCRIBE \N • CON ` ` SE ESCRIBE SALTANDO DE LÍNEA

La ficha de un producto

```
const ficha = `  
  Producto : ${macbook.nombre}  
  Categoría: ${macbook.categoria}  
  Precio   : ${formatearPrecio(macbook.precio)}  
  Con IGV  : ${formatearPrecio(precioConIgv(macbook.precio))}  
  Stock    : ${macbook.stock} unidades  
`;
```

Lo que se ve es lo que sale, **sangría incluida**. Y esta es exactamente la forma en que se arma un pedazo de HTML desde JavaScript: la **Clase 9**.

EN JAVA NO EXISTE · A LA IZQUIERDA DEL = VAN LAS PROPIEDADES QUE QUIERO

Desestructurar un objeto

ANTES · UNA LÍNEA POR PROPIEDAD

```
const nombre = producto.nombre;  
const precio = producto.precio;  
const stock  = producto.stock;
```

HOY · UNA LÍNEA

```
const { nombre, precio, stock } = producto;
```

Esas llaves **no arman** un objeto: lo **abren**. Llaves a la izquierda del =, abren. A la derecha, arman.

TRES COSAS QUE SE LE PUEDEN PEDIR A LAS MISMAS LLAVES

Renombrar, **anidar** y recoger

```
const { nombre: titulo  
} = p;
```

Renombrado. A la izquierda de los dos puntos, la **llave del objeto**. A la derecha, **mi variable**. No al revés.

```
const { envio: { zona  
} } = p;
```

Anidada. Un objeto dentro de otro se abre con otro par de llaves. Solo si el de adentro existe.

```
const { id, ...resto }  
= p;
```

Con rest. Saco una propiedad y todo lo demás queda junto en un objeto nuevo.

Y el **valor por defecto** con un `=`: `const { descuento = 0 } = p;` entra solo si la propiedad **no existe** o vale `undefined`.

CON OBJETOS MANDA EL NOMBRE · CON ARRAYS MANDA LA POSICIÓN

Desestructurar arrays

```
const [primera, segunda] = categorias; // por posición
const [, , tercera] = categorias; // una coma sola salta un lugar
const [principal, ...demas] = categorias; // el primero aparte, el resto junto
[primero, segundo] = [segundo, primero]; // intercambiar, sin variable auxiliar
```

Aquí el **nombre lo elijo yo**, porque un arreglo no tiene llaves: tiene lugares. Lo que decide el valor es **dónde** escribí la variable.

EL USO QUE MÁS VAN A ESCRIBIR EN SU VIDA

Desestructurar en el **parámetro**

ANTES · EL OBJETO ENTERO, Y ADENTRO SE ABRE

```
const etiqueta = (p) => p.nombre + " " + p.precio;
```

HOY · SE ABRE EN EL PROPIO PARÉNTESIS

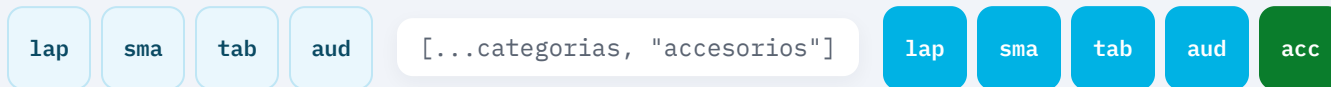
```
const etiqueta = ({ nombre, precio }) => `${nombre} ${precio}`;
```

La firma pasa a ser **documentación**: leyendo el paréntesis se sabe qué necesita la función. Y lo que no pide, no le importa.

Así reciben sus props los componentes de React. En la semana 5 van a escribir esto decenas de veces por día. Hoy lo aprendemos con productos.

... DESARMA UN ARREGLO Y SUELTA SUS ELEMENTOS AHÍ DONDE ESTÉ ESCRITO

Spread: copiar, agregar, combinar



Sale un arreglo **nuevo** y el original queda **intacto**. Es el reemplazo de push y de unshift, y la forma corta de la copia que sort necesita.

En React esto es una regla, no un gusto: el estado no se modifica, se reemplaza. Lo que practicamos hoy es la forma de trabajo de las próximas veinte clases.

HAY FUNCIONES QUE NO RECIBEN UN ARREGLO: RECIBEN ARGUMENTOS SUELTOS

Spread al llamar

Math.max(precios)

→ NaN

Le pasé **un arreglo** donde esperaba números sueltos.
No sabe qué hacer con él.

Math.max(...precios)

→ 1999.99

Los tres puntos **desarman** el arreglo en argumentos separados por coma.

Es el mismo desarmar de la lámina anterior, en otro sitio: dentro del **paréntesis de la llamada**.

LO QUE MÁS CONFUNDE DEL DÍA: LOS MISMOS TRES PUNTOS, AL REVÉS

Rest: juntar lo que llegue

AL LLAMAR → SPREAD → DESARMA

```
sumar(...precios)
```

un arreglo se convierte
en argumentos sueltos



AL DECLARAR → REST → JUNTA

```
(...precios) => ...
```

los argumentos sueltos
se convierten en un arreglo

Para no confundirse: si es algo que estoy **armando**, los puntos **desparraman**. Si es algo que estoy **recibiendo**, los puntos **recogen**. El parámetro rest va **al final** y solo puede haber uno.

CON OBJETOS, LOS PUNTOS SUELTAN PROPIEDADES

Spread en **objetos**

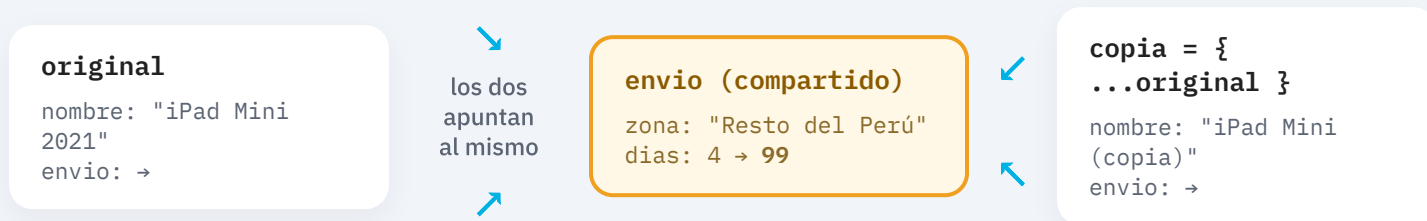
```
const enOferta = { ...producto, precio: 1799.99 }; // ✓ el precio nuevo gana  
const enOferta = { precio: 1799.99, ...producto }; // ✗ el spread pisó el precio nuevo
```

Gana la última propiedad que se escribe. Por eso el spread va **primero** y lo que quiero cambiar va después.

Al devolver un objeto desde una flecha corta, **los paréntesis son obligatorios**: (producto, precio) => ({ ...producto, precio }). Sin ellos, JavaScript lee el { como el cuerpo de la función.

EL LÍMITE QUE HAY QUE CONOCER SÍ O SÍ

La copia es superficial



El **primer nivel** se copia de verdad: el nombre es propio de cada uno. Lo que está **adentro** se comparte. Solución: `{ ...original, envio: { ...original.envio } }`

TRES DETALLES CORTOS QUE APARECEN EN TODO EL CÓDIGO MODERNO

Abreviadas, calculadas y defaults

{ unidades }

Propiedad abreviada. Si la propiedad se va a llamar igual que la variable, se escribe una vez. Reemplaza a { unidades: unidades }.

{ [campo]: valor }

Nombre calculado. El nombre de la propiedad sale de un dato, así que va entre corchetes. Sin ellos, la propiedad se llamaría campo.

(precio, pct = 10) =>

Parámetro por defecto. Entra cuando el argumento falta o vale undefined. Un 0 de verdad se respeta.

El truco viejo era `pct = pct || 10` en la primera línea. Tapa **todo** lo falso, así que un descuento de 0 se convertía en 10. Eso es un error de facturación que nadie ve hasta el reclamo.

?. y ??, la pareja

?.

`cupon?.porcentaje`

Encadenamiento opcional. Si lo de la izquierda es null o undefined, devuelve undefined y **sigue**, en vez de detener el archivo.

??

`cupon?.porcentaje ?? 0`

Fusión de nulos. Da lo de la derecha **solo** si lo de la izquierda es null o undefined. El `||` tapa además el 0, el "" y el false.

Se escriben casi siempre juntos y en ese orden: **preguntar sin romper**, y después **poner un respaldo**. Es lo que hace que el código aguante datos incompletos, que es lo que va a llegar de una API en dos clases.

NO LOS CODEAMOS: LOS LEEMOS PARA RECONOCERLOS CUANDO PASEN

Errores frecuentes

- 1 Comillas normales en vez de invertidas.** `"Hola ${nombre}"` imprime `Hola ${nombre}` tal cual. Sin error y sin línea roja.
- 2 Renombrar al revés.** En `{ nombre: titulo }`, la **izquierda** es la llave del objeto. Al revés, la variable queda en `undefined`.
- 3 Desestructurar algo indefinido:** `Uncaught TypeError: Cannot destructure property 'nombre' of '...' as it is undefined.` Preguntar antes con `?.` o `??`.
- 4 Creer que el spread copia en profundidad.** Copia **un nivel**. Si algo cambia "solo" y nadie tocó el original, el sospechoso es este.
- 5 Poner el spread al final.** `{ precio: 500, ...producto }` no cambia nada: gana la última propiedad escrita.
- 6 Olvidar `type="module"`:** `Uncaught SyntaxError: Cannot use import statement outside a module.` No corre nada.
- 7 Olvidar la extensión** (`Failed to load resource: ... 404 (Not Found)`) **o abrir con doble clic** (blocked by CORS policy). Los módulos viajan por http.
- 8 Confundir `export default` con `export con nombre`:** The requested module `'./formato.js'` does not provide an export named `'formatearPrecio'`. Sin llaves.

LA TIENDA HACE LO MISMO QUE AYER · LO QUE CAMBIÓ ES CÓMO ESTÁ ESCRITA

Resumen

`` ` y ${ }`

interpolación, con cualquier expresión adentro

`texto multilínea`sin `\n` y sin pegar pedazos`{ a, b } = obj`

abrir un objeto en una línea

`{ a: otro, b = 0 }`

renombrar y valor por defecto

`{ envio: { zona } }`

anidada, si el de adentro existe

`[a, , c] = arr`

por posición, saltando lugares

`[a, b] = [b, a]`

intercambiar sin variable auxiliar

`({ nombre }) =>`

en el parámetro: así recibe props React

`[...arr, x]`

copiar, agregar y combinar sin mutar

`{ ...obj, precio }`

actualizar: gana el último escrito

`copia superficial`

un nivel; lo de adentro se comparte

`(...args) =>`

rest: junta lo que llegue

`{ x }, { [k]: v }`

abreviadas y calculadas

`?. y ??`

preguntar sin romper, y respaldar

`export / default`

lo que sale del archivo

`type="module"`

y la página se abre con servidor

CON LA GRABACIÓN AL LADO · LA QUE NADIE LOGRE LA VEMOS JUNTOS LA PRÓXIMA

5 tareas

FÁCIL

Replicar la clase y sumar un producto. Los cuatro archivos y el `type="module"`. Después, un producto nuevo con `agregarProducto` y su ficha en un template literal de varias líneas. Comprobar: 7 y 6.

INTER. 1

Un módulo más. Mover `tarifaPorZona` y `costoDeEnvio` a `js/envio.js`, con sus `export` y el `import` de `ENVIO_GRATIS_DESDE`. La consola tiene que imprimir lo mismo.

INTER. 2

La ficha corta. `fichaCorta` con el producto desestructurado **en el paréntesis**, incluido el envío anidado, y un template literal de tres líneas. Probar con `map`.

DIFÍCIL 1

Agrupar por categoría. `agruparPorCategoria(items)`: cada llave una categoría, cada valor el **arreglo** de sus productos. `reduce` + `spread` + nombre calculado + `??`.

DIFÍCIL 2

Rebajar una categoría sin mutar nada. `aplicarDescuentoA(items, categoria, porcentaje = 10)`. Comprobar con `===`: los no tocados dan `true`, los tocados `false`.

Próxima clase · el DOM

La tienda cobra vida

Buscar un elemento de la página, cambiarle el texto, crear elementos nuevos y meterlos adentro. Las cuatro tarjetas escritas a mano se borran y el catálogo se pinta **solo**, desde el arreglo de **datos.js**. Las seis, no las cuatro.

¿Cómo hacemos que esto salga en **la página** y no en la consola?

Y todo lo de hoy va a estar ahí: la plantilla de cada tarjeta será un **template literal de varias líneas**, cada tarjeta recibirá el producto **desestructurado en el parámetro**, y la función que las arma vivirá en **su propio módulo**.

Nos vemos la próxima clase